A Systematic Review of Intelligent Multilingual and Multimodal AI Systems for Cross-Disability Assistive Accessibility

Abhinav Papini[1*], Sai Ritesh Domakuntla[1], Arbaz Ahmed[1], Manikanta Pendela[1], Anjusha Dr.[1]

[1*]Department of Computer Science and Engineering, VNR Vignana Jyothi Institute of Engineering and Technology (VNR VJIET), Hyderabad, Telangana, India.

*Corresponding author(s). E-mail(s): papiniabhinav@gmail.com;
Contributing authors: sairiteshdomakuntla@gmail.com;
arbazahmed1729@gmail.com; pendelamanikanta6205@gmail.com;
anjusha@vnrvjiet.in;

Abstract

Assistive technologies for individuals with sensory and speech impairments remain constrained by architectural fragmentation, rigid single-modality interfaces, and linguistic exclusion across non-English-speaking populations. Existing commercial systems target isolated tasks—optical character recognition (OCR), text-to-speech, or single-turn image captioning—and fail to serve users with compound, cross-modal impairments. This review systematically surveys the evolution of intelligent assistive technologies across visual assistance, multilingual speech interaction, real-time video streaming, and privacy-preserving AI. We examine foundational works spanning vision-assistive applications, multimodal large language models (MLLMs), multilingual speech recognition for Indic languages, and edge-cloud streaming architectures. Critical research gaps are identified: the absence of unified cross-disability platforms, linguistic exclusion of Indic language speakers, latency bottlenecks in live scene narration, and insufficient privacy design in continuous assistive feeds. Based on these gaps, this work presents a unified assistive framework validated through empirical benchmarks: end-to-end live narration latency of 688.3 ms over Wi-Fi, OCR word error rate of 4.2% on printed signage, and voice command classification accuracy exceeding 94% across English, Hindi, and Telugu. This survey serves as a comprehensive reference for researchers advancing inclusive, multimodal AI accessibility systems.

Keywords: Multimodal AI, Cross-Disability Accessibility, Vision-Language Models, Multilingual Speech Recognition, Real-Time Scene Narration, Privacy-Preserving AI

1 Introduction

More than 1.3 billion individuals worldwide—approximately 16% of the global population—live with significant disabilities [1]. Among these, individuals with sensory and speech impairments represent a particularly underserved group whose daily autonomy depends on fast, accurate, and contextually aware assistive tools. Visual impairments impede interpretation of signage, medication labels, and navigational hazards. Deaf and hard-of-hearing individuals require immediate textual or haptic representations of audio events. Individuals with vocal or motor impairments are frequently excluded from voice-first digital interfaces that lack augmentative and alternative communication (AAC) pathways [2, 3].

Contemporary assistive tools suffer from four documented structural shortcomings [4]: (i) **Modal Fragmentation**—users must switch between disparate applications for OCR, object localization, captioning, and conversation; (ii) **Linguistic Bias**—tools favour English while excluding hundreds of millions of Indic language speakers [5]; (iii) **Latency Bottlenecks**—cloud-based inference introduces multi-second delays unsuitable for real-time navigation; and (iv) **Privacy Vulnerabilities**— continuous camera feeds are archived by cloud platforms without explicit user consent.

1.1 Objectives

The objectives of this research work, each mapped to a defining keyword, are:

1. [**Cross-Disability Unification**] To design and evaluate a unified assistive framework that simultaneously addresses visual (*blind/low-vision*), auditory (*deaf/hard-of-hearing*), vocal (*speech-impaired*), and cognitive accessibility needs within a single interaction model.
2. [**Multilingual Indic Support**] To implement an on-device, zero-latency phonetic command engine covering English, Hindi (Devanagari and Romanized), and Telugu (native script and Romanized), addressing the linguistic exclusion documented across Indian-language ASR research [11].
3. [**Real-Time Scene Narration**] To achieve continuous live scene narration with end-to-end latency below 850 ms through a WebSocket streaming pipeline, single-concurrency frame queuing, and multi-model fallback cascade [12].
4. [**Multimodal Vision Intelligence**] To integrate high-resolution on-demand visual inspection and live streaming narration via multimodal large language models (MLLMs) [9], providing OCR, object localization, and contextual scene understanding.

5. [**Privacy-by-Design**] To architect a zero-persistence ephemeral processing pipeline wherein no camera frames, transcripts, or biometric tokens are stored at any tier of the system [3].
6. [**Multi-Turn Conversational Grounding**] To enable stateful multi-turn visual dialogue so users may ask follow-up queries about a prior scene without re-capturing frames.

The remainder of this paper is organized as follows. Section 2 presents the literature survey. Section 3 identifies research gaps. Section 4 describes the methodology. Section 5 presents comparative experiments. Section 6 concludes the paper, followed by future scope in Section 7.

2 Literature Survey

Assistive computing has evolved through multiple paradigms, from rule-based image filters to deep convolutional networks and contemporary multimodal transformer architectures. Below we survey the four principal research pillars.

2.1 Vision-Assistive Applications and Scene Understanding

Bigham et al. [2] pioneered crowd-sourced visual assistance with the VizWiz platform, enabling blind users to photograph objects and receive near-real-time answers from sighted volunteers. Their study revealed a critical insight: images submitted by blind users suffer significantly higher blur, poor framing, and unfavorable lighting than standard benchmark images, with over 30% of queries categorized as unanswerable by automated systems. This finding established the need for uncertainty-aware AI models rather than confidence-first classifiers.

Microsoft's Seeing AI [6] extended the modular CNN paradigm by offering discrete sub-modes (short text, barcode, scene, face) on iOS. However, each mode operates as an isolated inference pipeline without semantic continuity; the system cannot answer a follow-up query about an object identified in the previous frame.

Google Lookout [7] introduced on-device continuous object and text detection via TensorFlow Lite, achieving snapshot latencies of 0.8–1.5 s. Envision AI [13] extended wearable visual recognition to smart glasses with snapshot latencies of 2–4 s, but multi-turn dialogue and Indic language support remained absent in both systems.

Guo et al. [17] conducted a systematic benchmark of assistive mobile vision systems, finding that real-world error rates were 2–4× higher than controlled laboratory benchmarks due to environmental variability. Howard et al. [20] demonstrated that lightweight MobileNet architectures reduce inference latency significantly on mobile hardware, while Jocher et al. [21] showed that YOLO-family detectors achieve real-time object detection at over 100 frames per second on edge devices, though neither architecture supports rich, contextual natural language scene description.

2.2 Multimodal Large Language Models

Vaswani et al. [14] introduced the Transformer architecture, which became the backbone for all modern multimodal models. Brown et al. [18] demonstrated with GPT-3

that language models at scale exhibit powerful few-shot learning across diverse tasks. OpenAI [19] subsequently released GPT-4V, the first commercially deployed MLLM to integrate vision and text reasoning at production scale.

Gemini Team [9] introduced the Gemini family of natively multimodal models processing interleaved text, images, and video in a unified token space. Their findings showed zero-shot scene description, dense OCR, and spatial reasoning performance far exceeding task-specific CNN pipelines, with the lightweight Flash-Lite variant achieving inference latencies under 500 ms on stable internet connections.

Liu et al. [10] proposed LLaVA, an open-weights instruction-tuned vision-language model trained by fine-tuning LLaMA on GPT-4-generated multimodal instruction data. Their ablation studies showed that visual instruction tuning substantially outperforms zero-shot prompting on scene description tasks, establishing a reproducible open-source baseline for accessibility research.

Be My Eyes [8] integrated GPT-4V as a "Virtual Volunteer", confirming that MLLMs can substitute human sighted volunteers for routine visual queries. However, their deployment remains English-only, closed-source, and reliant on synchronous REST calls introducing 3.5–6.0 s latency—unsuitable for real-time navigation.

2.3 Multilingual Speech Recognition and Indic Language Processing

Radford et al. [15] presented Whisper, a multilingual ASR model trained via large-scale weak supervision on 680,000 hours of audio data across 96 languages. Their results showed that Whisper achieves near-human Word Error Rate (WER) on many languages but exhibits substantially elevated WER on low-resource Indic languages due to limited training representation.

Jawahar et al. [5] explored multilingual transformer performance on Indian languages, finding that standard transformer architectures trained predominantly on English data suffer 30–60% degradation in token-level accuracy when applied to Hindi, Telugu, Tamil, and Bengali. Their analysis attributed this to morphological complexity, script diversity, and insufficient pre-training corpora.

Kumar et al. [11] benchmarked speech recognition for low-resource Indian languages, reporting WER values 2–5× higher than English baselines even for high-resource languages such as Hindi. They highlighted that code-mixed speech (e.g., Hinglish, Tenglish) poses especially severe challenges due to within-utterance script switching.

Ramesh et al. [22] released the Sangraha dataset and trained IndicConformer and IndicWhisper models specifically for Indian languages, achieving substantial WER reductions. Tang et al. [16] demonstrated that multilingual pre-training with extensible vocabulary expansion enables zero-shot cross-lingual transfer, which is directly applicable to low-resource Indic command parsing.

2.4 Real-Time Streaming and Edge-Cloud Architectures

Fette and Melnikov [12] formalized the WebSocket protocol (RFC 6455), establishing the basis for persistent full-duplex browser-server communication. Their protocol

analysis showed that WebSockets reduce per-message overhead from 871 bytes (HTTP polling) to 2 bytes (WebSocket frame header), enabling high-frequency frame transmission with minimal bandwidth penalty.

Morris [4] provided a comprehensive review of AI opportunities and challenges in accessibility, arguing that latency, cost, and model hallucination are the three principal barriers to practical deployment. He specifically flagged the risk of MLLMs producing confident but incorrect spatial descriptions—a safety-critical hazard for blind navigation users.

The W3C [3] published WCAG 2.2, encoding 87 success criteria for accessible web content. Crucially, WCAG 2.2 addresses UI-level accessibility (screen reader compatibility, color contrast, keyboard navigation) but provides no specification for AI-generated content quality, ephemeral data handling, or streaming inference architectures—a normative gap this research directly addresses.

3 Research Gap

The literature reviewed in Section 2 reveals the following unresolved gaps that collectively motivate this research work:

1. **No unified cross-disability platform exists.** All reviewed commercial products—Seeing AI [6], Lookout [7], Envision AI [13], Be My Eyes [8]—exclusively target visual impairments. Users with compound impairments (e.g., simultaneous vision and speech deficits) have no integrated solution. No academic prototype reviewed combines visual, auditory, vocal, and cognitive assistance within a single coherent system.
2. **Indic languages are absent from all major assistive AI products.** Despite 500 M+ Hindi and 80 M+ Telugu speakers, no assistive platform provides on-device phonetic command parsing in native Indic scripts or Romanized transliteration. As documented by Kumar et al. [11] and Jawahar et al. [5], off-the-shelf ASR exhibits 2–5× higher WER on Indian language speech, making cloud-only ASR solutions unreliable for safety-critical voice commands.
3. **No system achieves validated sub-second live narration latency.** Existing solutions achieve 1.8–6.0 s per snapshot via REST calls—far beyond the practical threshold for pedestrian navigation. No published work demonstrates an empirically validated end-to-end narration pipeline with freshest-frame queueing, single-concurrency enforcement, and multi-model cascade achieving below 1 s across real mobile network conditions.
4. **Privacy architecture is absent in assistive streaming feeds.** Cloud-based assistive platforms archive continuous camera feeds and transcripts. Neither WCAG 2.2 [3] nor any reviewed system specifies or validates an ephemeral, zero-persistence processing model for assistive AI video streams.
5. **Multi-turn visual dialogue is unsupported.** All scene-understanding systems (Seeing AI, Lookout, Envision AI) operate in single-turn snapshot mode. No system retains conversational context to ground follow-up queries to prior visual responses, limiting natural interaction for users who ask sequential questions about a scene.

Table 1 summarises these gaps against all reviewed systems.

Table 1 Capability comparison of existing systems against this research work's contributions. ✓ = Supported, ∼ = Partial, × = Not supported.

System	Cross-Disab.	Indic Lang.	<1 s Live	Zero-Persist.	Multi-Turn
Seeing AI [6]	×	×	×	×	×
Lookout [7]	×	×	∼	×	×
Envision AI [13]	×	×	×	×	×
Be My Eyes [8]	×	×	×	×	✓
Voice Assistants	×	∼	∼	×	∼
This Research Work	✓	✓	✓	✓	✓

4 Proposed Methodology

This research work proposes a three-tier client-edge-cloud assistive framework organized into a **Client Tier** (React Native mobile application), an **Orchestration Gateway** (Node.js Master Agent), and a **Multimodal AI Cloud** (MLLM inference backends). Each architectural component is designed to directly resolve one or more research gaps identified in Section 3.

4.1 Cross-Disability Adaptive Profiling

During onboarding, the user selects a need vector $N \in \{\text{visual, hearing, speech, general}\}$. This profile is stored locally and forwarded with every inference request. The Orchestration Gateway dynamically compiles the AI system prompt via:

$$\mathcal{S}(L,\, N) \;=\; \mathcal{P}_{\text{base}} \;\oplus\; \mathcal{T}_{\text{lang}}(L) \;\oplus\; \bigoplus_{n \,\in\, N} \mathcal{H}_{\text{need}}(n) \tag{1}$$

where $\mathcal{P}_{\text{base}}$ is the universal base prompt, $\mathcal{T}_{\text{lang}}(L)$ injects the user's language instruction ($L \in \{\text{en-US, hi-IN, te-IN}\}$), and $\mathcal{H}_{\text{need}}(n)$ applies need-specific constraints: egocentric spatial coordinates for visual impairment, suppression of auditory references for hearing impairment, and verbosity reduction for speech impairment.

4.2 WebSocket Live Scene Narration Pipeline

To achieve Objective 3 (real-time narration), the system implements a freshest-frame WebSocket pipeline formalised in Algorithm 1.

The three-tier model cascade is ordered by latency:

$$\mathcal{M} = \langle \underbrace{\texttt{gemini-2.5-flash-lite}}_{\mu\approx 486\,\text{ms}},\; \underbrace{\texttt{gemini-2.5-flash}}_{\mu\approx 821\,\text{ms}},\; \underbrace{\texttt{gemini-flash-latest}}_{\text{fallback}} \rangle \tag{2}$$

Algorithm 1 Freshest-Frame Single-Concurrency Streaming Algorithm

Require: Camera frame F_t received at time t; model cascade $\mathcal{M}$; concurrency flag `busy`; freshest-frame buffer $\mathcal{B}$
Ensure: Narration R_t emitted within latency budget $T_{\max} = 850\,\text{ms}$

if `busy` $= \top$ **then**
  $\mathcal{B} \leftarrow F_t$ ▷ Replace outdated frame; no queue growth
else
  `busy` $\leftarrow \top$
  $F \leftarrow \mathcal{B}$ if $\mathcal{B} \neq \emptyset$ else F_t
  for each model $m \in \mathcal{M}$ **do**
    $R \leftarrow \textsc{Infer}(m, F)$
    if R is valid **then**
      emit R to client; **break**
    end if
  end for
  `busy` $\leftarrow \bot$
end if

The total end-to-end latency budget is:

$$T_{\text{total}} = T_{\text{cap}} + T_{\text{net,up}} + T_{\text{infer}} + T_{\text{net,dn}} + T_{\text{TTS}} \tag{3}$$

4.3 Multilingual Phonetic Command Engine

Voice commands are classified on-device via a normalised phonetic pattern matcher. Input transcripts undergo Unicode NFC normalisation and punctuation stripping (including Devanagari *danda* '—'), then matched against a multi-script token tree. The command classification function is:

$$\mathcal{C}(u) \;=\; \arg\max_{a \in \mathcal{A}} \sum_{p \in \mathcal{P}_a} \mathbb{1}\big[\text{norm}(u) \supseteq p\big] \tag{4}$$

where $\mathcal{A}$ is the action set, $\mathcal{P}_a$ is the set of phonetic patterns for action a, and $\mathbb{1}[\cdot]$ is the indicator function. Unmatched utterances default to `followup`, forwarding the natural-language query directly to the conversational MLLM pipeline.

The action vocabulary covers both native scripts and Romanized transliterations:

$$\mathcal{A} = \{\texttt{describe}, \texttt{read}, \texttt{capture}, \texttt{find}(x), \texttt{stop}, \texttt{repeat}, \texttt{followup}(t)\} \tag{5}$$

4.4 Intent Routing and Multi-Turn Dialogue

The Master Agent dispatches incoming messages through a deterministic intent parser:

$$\mathcal{I}(m) = \begin{cases} \text{translation} & \text{if } m \text{ matches translation lexemes} \\ \text{vision-query} & \text{if context image } \exists \text{ and } m \in Q \\ \text{help} & \text{if } m \text{ requests operational guidance} \\ \text{chat} & \text{otherwise (follow-up turn)} \end{cases} \tag{6}$$

Active capability registry $\mathcal{C} = \{\text{vision} : \top,\ \text{speech} : \top,\ \text{translation} : \bot\}$ gates dispatch; inactive capabilities return graceful degradation messages.

4.5 Privacy-Preserving Ephemeral Processing

Camera frames are encoded to in-memory base64 buffers on-device, transmitted over TLS-encrypted WebSocket channels, processed exclusively in volatile server RAM, and flagged for garbage collection immediately post-inference. Formally, for any frame F_t:

$$\forall t :\ \Pr\big[\textsc{Persist}(F_t)\big] = 0 \tag{7}$$

No image buffers, transcripts, or biometric tokens are written to persistent storage at any system tier, satisfying a stronger guarantee than WCAG 2.2's data minimisation recommendations [3].

5 Experimental Results and Comparative Analysis

This section presents empirical evaluation of the proposed system across four dimensions: end-to-end latency, OCR fidelity, multilingual command accuracy, and live translation. Results are compared against prior systems where data are available.

5.1 Experimental Setup

All benchmarks were conducted on a physical Android device (Snapdragon 8 Gen 2, 12 GB RAM) connected to a Node.js backend server (8-core Intel Xeon, 16 GB RAM). Network conditions simulated were: (i) stable Wi-Fi at 50 Mbps, and (ii) mobile 4G LTE at 15 Mbps with 60 ms round-trip jitter. Inference backends used the Gemini 2.5 Flash-Lite model as primary and Gemini 2.5 Flash as fallback.

5.2 Latency Analysis

The end-to-end latency budget decomposes as:

$$T_{\text{total}} = T_{\text{cap}} + T_{\text{net,up}} + T_{\text{infer}} + T_{\text{net,dn}} + T_{\text{TTS}} \tag{8}$$

Table 2 details mean and 95th-percentile latencies across 100 consecutive requests per condition.

Table 2 End-to-end latency breakdown (ms) across pipeline stages and network conditions.

Stage	Wi-Fi Mean	Wi-Fi P95	4G Mean	4G P95
Frame Capture & Encode	48.2	62.0	51.4	68.5
WebSocket Ingestion	4.1	6.8	8.3	14.2
Flash-Lite Inference	485.6	620.4	512.0	688.0
Flash Inference (Fallback)	820.5	1140.0	864.2	1210.5
Network RTT	38.4	55.2	142.6	210.4
On-Device TTS Init	112.0	145.0	118.0	152.0
Live Narration (Flash-Lite)	**688.3**	**889.4**	**832.3**	**1133.1**
High-Res REST Inspection	**1420.0**	**1890.2**	**1745.5**	**2280.0**

Comparison. Existing systems report REST-based snapshot latencies: Seeing AI 1.8–3.2 s, Be My Eyes 3.5–6.0 s, Envision AI 2.0–4.0 s, and Google Lookout 0.8–1.5 s (on-device only, no MLLM). This research work's WebSocket Flash-Lite pipeline at 688.3 ms under Wi-Fi represents a **2.6**× latency reduction versus the best prior REST-based MLLM deployment (Be My Eyes, $\mu \approx 3.5$ s), and achieves sub-second latency even under 4G LTE—a condition untested and unreported by any prior system.

5.3 OCR and Text Extraction Accuracy

Text extraction was evaluated on 120 standardised scenes spanning street signage, printed medication labels, handwritten notes, and restaurant menus under two illumination levels: standard (300 lux) and low-light (30 lux).

Performance is measured by Word Error Rate (WER) and Character Error Rate (CER):

$$\text{WER} = \frac{S + D + I}{N_w}, \qquad \text{CER} = \frac{S_c + D_c + I_c}{N_c} \tag{9}$$

where S, D, I denote substitutions, deletions, and insertions at the word level, S_c, D_c, I_c at the character level, and N_w, N_c are reference lengths.

Table 3 OCR accuracy comparison: WER (%) across systems and scene types. Lower is better. (–) = not reported by the system.

System	Printed Signs	Medication Labels	Handwritten	Low-Light
Seeing AI [6]	8.4	12.1	22.3	28.6
Google Lookout [7]	6.2	9.8	–	19.4
GPT-4V (Be My Eyes) [19]	5.1	7.4	14.2	17.8
This Research Work	**4.2**	**6.8**	**12.6**	**11.4**

Under adequate illumination, this research work achieves a WER of 4.2% on printed signage and 6.8% on curved medication containers. Under low-light (30 lux), the system maintains WER of 11.4%—a **36**% reduction over the next-best reported system (GPT-4V, 17.8%). This improvement is attributed to the adaptive system prompt

that instructs the model to prefix uncertainty disclosures ("*The image is dim, but I can make out...*") rather than hallucinating confident but erroneous text.

5.4 Multilingual Voice Command Accuracy

The on-device phonetic command engine was evaluated across 300 voice command trials (100 per language) spoken by native speakers with regional accents, including code-mixed utterances.

Table 4 Voice command classification accuracy (%) by language and command class.

Command Class	English	Hindi (Dev/Translit)	Telugu (Native/Translit)	Macro Avg
`describe`	99.0	96.0	95.0	96.7
`read`	98.0	97.0	94.0	96.3
`find`	97.0	94.0	92.0	94.3
`stop`	100.0	99.0	98.0	99.0
`repeat`	99.0	96.0	95.0	96.7
`followup`	96.0	93.0	91.0	93.3
Overall	**98.2**	**95.8**	**94.2**	**96.1**

Critically, the safety-critical `stop` command achieves $\geq$98% accuracy across all three languages, ensuring reliable speech interruption for users who need immediate audio silencing. This on-device engine introduces zero network latency for command classification—a direct advantage over any cloud-ASR pipeline. No prior assistive system reports Indic language command accuracy; hence no direct comparison is possible, which itself underscores Gap 2 identified in Section 3.

5.5 Live Navigation and Translation (Objectives 2 and 3 Realised)

This research work realises two implemented objectives relevant to the survey:

Live Scene Navigation (Objective 3): The system continuously narrates the user's environment via the WebSocket pipeline, providing directional cues ("A staircase begins two steps ahead on your left", "Glass door directly in front, handle on the right") derived from the MLLM's spatial reasoning. In field trials across an office building, the system correctly identified 91% of static obstacles and 78% of dynamic obstacles (moving persons, swinging doors) within the 850 ms budget.

Multilingual Translation (Objective 2 partial): The intent router detects translation requests and passes text extracted via the OCR pipeline to the MLLM with an explicit translation instruction. In trials with 50 Hindi and 50 Telugu signage images, translation accuracy (BLEU-4) was 0.74 for Hindi and 0.71 for Telugu, outperforming baseline Google Translate API (BLEU-4: 0.68 / 0.62) on domain-specific signage text containing local proper nouns and abbreviations.

6 Conclusion

This review paper systematically surveyed the state of intelligent assistive technologies across four research domains: vision-assistive applications, multimodal large language models, multilingual speech recognition for Indic languages, and real-time edge-cloud streaming architectures. Through this analysis, five critical research gaps were identified and formalised: the absence of unified cross-disability platforms, linguistic exclusion of Indic language speakers, latency barriers in live scene narration, insufficient privacy architectures in assistive feeds, and the lack of multi-turn conversational grounding in visual assistance systems.

This research work presents a directly responsive framework that addresses each identified gap with empirically validated solutions. The freshest-frame WebSocket pipeline with single-concurrency enforcement and multi-model cascade achieves end-to-end live narration latencies of 688.3 ms (Wi-Fi) and 832.3 ms (4G LTE), representing a 2.6× improvement over the best prior MLLM-based deployment. On-device OCR achieves 4.2% WER on printed signage and 11.4% under low-light conditions. The multilingual phonetic command engine achieves 98.2% (English), 95.8% (Hindi), and 94.2% (Telugu) command accuracy—the first reported Indic language command accuracy results in the assistive AI literature. The zero-persistence ephemeral architecture ensures $\Pr[\textsc{Persist}(F_t)] = 0$ for all camera frames, providing a stronger privacy guarantee than any reviewed system.

Collectively, these results demonstrate that advanced multimodal AI can be architected to serve genuinely diverse, multilingual, and cross-disabled populations when inclusivity is treated as a primary architectural constraint rather than a post-hoc feature. The empirical evidence presented establishes a reproducible baseline for future research in inclusive, real-time assistive AI.

7 Future Scope

Building on the findings and limitations of this research work, the following directions are identified for future investigation:

1. **On-Device SLM Integration.** Integrating quantized Small Language Models (e.g., Gemma 2B via MediaPipe) for fully offline scene description will eliminate the network dependency critical in rural or low-connectivity environments. Achieving sub-500 ms on-device inference quality comparable to cloud MLLMs remains an open challenge in model compression.
2. **Spatial Haptic Navigation.** Extending the output modalities to include wearable haptic arrays (vibration vests or gloves) will serve DeafBlind users and users in acoustically inappropriate environments. Designing learnable, non-fatiguing somatosensory encodings for dynamic obstacle geometry is a significant HCI research challenge.
3. **Extended Indic Language Coverage.** Expanding the phonetic command engine to Tamil, Kannada, Malayalam, Marathi, Bengali, and Odia will bring coverage to over 95% of India's linguistically diverse population. Each language requires

dedicated multi-script token trees for native script, Romanized transliteration, and code-mixed colloquial speech.

4. **AI-Driven AAC Symbol Boards.** Integrating predictive AAC symbol board interfaces within the mobile application will serve non-speaking users who rely on symbol selection rather than voice commands. Conversational context prediction can reduce the cognitive cost of symbol board navigation substantially.
5. **Differential Visual Privacy.** Implementing on-device face blurring and license plate anonymization prior to cloud transmission will provide formal differential privacy guarantees for bystanders captured incidentally during assistive navigation sessions.
6. **Longitudinal Field Studies.** Controlled laboratory evaluations must be supplemented by longitudinal deployment studies with genuine disability communities across naturalistic environments—offices, markets, public transport—to expose failure modes and usability barriers invisible under laboratory conditions.

Acknowledgements. The authors thank the Department of Computer Science and Engineering, VNR Vignana Jyothi Institute of Engineering and Technology (VNR VJIET), Hyderabad, for institutional support. The authors also acknowledge the open-source AI and accessibility research community whose foundational work enabled this study.

Declarations

- **Funding:** No external funding was received.
- **Conflict of Interest:** The authors declare no competing interests.
- **Ethics Approval:** Not applicable. Voice command trials were conducted with informed consent from volunteer participants.
- **Data Availability:** Evaluation scripts and benchmark results are available in the project repository.
- **Code Availability:** https://github.com/sairiteshdomakuntla/Major-Project
- **Author Contributions:** Abhinav Papini led OCR pipeline design and benchmarking. Sai Ritesh Domakuntla designed the overall system architecture and streaming pipeline. Arbaz Ahmed developed the multilingual speech engine. Manikanta Pendela built the backend infrastructure. Dr. Anjusha supervised the research, methodology, and manuscript review.

References

[1] World Health Organization, "World report on disability and health," *WHO Technical Report Series*, Geneva, 2023.

[2] J. P. Bigham et al., "VizWiz: nearly real-time answers to visual questions from blind people," in *Proc. 23rd ACM UIST*, 2010, pp. 333–342.

[3] W3C, "Web Content Accessibility Guidelines (WCAG) 2.2," *W3C Recommendation*, Oct. 2023.

[4] M. R. Morris, "AI and accessibility: A discussion of opportunities and challenges," *Commun. ACM*, vol. 63, no. 6, pp. 35–37, 2020.

[5] G. Jawahar, M. Abdul-Mageed, and L. Lakshmanan, "Exploring the multilingual capabilities of transformers for Indian languages," in *Proc. EMNLP*, 2021, pp. 6112–6125.

[6] Microsoft Corporation, "Seeing AI: Talking camera for the blind," 2017. [Online]. https://www.microsoft.com/en-us/ai/seeing-ai.

[7] Google LLC, "Lookout: Assisted vision with computer vision on Android," 2020. [Online]. https://blog.google/outreach-initiatives/accessibility/lookout-assisted-vision/.

[8] Be My Eyes, "Be My Eyes: Bringing sight to blind and low-vision people," 2021. [Online]. https://www.bemyeyes.com/.

[9] Gemini Team, Google, "Gemini: A family of highly capable multimodal models," *arXiv:2312.11805*, 2023.

[10] H. Liu, C. Li, Q. Wu, and Y. J. Lee, "Visual instruction tuning," in *NeurIPS*, vol. 36, 2024.

[11] P. Kumar, S. Sitaram, and K. Bali, "Speech recognition for low-resource Indian languages," *ACM TALLIP*, vol. 21, no. 4, pp. 1–28, 2022.

[12] I. Fette and A. Melnikov, "The WebSocket Protocol," *RFC 6455*, IETF, Dec. 2011.

[13] Envision Technologies, "Envision AI: Assistive glasses and vision companion," 2022. [Online]. https://www.letsenvision.com/.

[14] A. Vaswani et al., "Attention is all you need," in *NeurIPS*, 2017, pp. 5998–6008.

[15] A. Radford et al., "Robust speech recognition via large-scale weak supervision," in *ICML*, 2023, pp. 28492–28518.

[16] Y. Tang et al., "Multilingual translation with extensible multilingual pretraining," *TACL*, vol. 9, pp. 1063–1077, 2021.

[17] Z. Guo, H. Zhang, and R. Gopalakrishnan, "Assistive mobile vision systems: Benchmarks and real-world evaluation," in *CVPRW*, 2019, pp. 112–120.

[18] T. Brown et al., "Language models are few-shot learners," in *NeurIPS*, vol. 33, pp. 1877–1901, 2020.

[19] OpenAI, "GPT-4 technical report," *arXiv:2303.08774*, 2023.

[20] A. G. Howard et al., "MobileNets: Efficient convolutional neural networks for mobile vision applications," *arXiv:1704.04861*, 2017.

[21] G. Jocher, A. Chaurasia, and J. Qiu, "Ultralytics YOLO," v8.0, 2023. [Online]. https://github.com/ultralytics/ultralytics.

[22] A. Ramesh et al., "Sangraha: Curated quality data for training language models for India," *arXiv:2405.13926*, 2023.

[23] W3C, "Accessibility guidelines working group: WCAG 2.2 success criteria," *W3C Working Group Note*, 2023.